

ZMQ&protobuf学习

一、ZMQ

学习目标

1. 理解 ZeroMQ 和 cppzmq 的基本概念。
2. 掌握使用 cppzmq 进行消息传递的基本方法。
3. 学会实现常见的消息模式，如请求-响应（REQ-REP）、发布-订阅（PUB-SUB）等。

学习内容

1. ZeroMQ 基础概念
2. cppzmq 入门
3. 常见的 ZeroMQ 模式（与 `cppzmq` 对应）
4. 高阶使用技巧与最佳实践

1 ZMQ基本介绍

1.1 介绍

ZeroMQ 是一个异步的消息传递库，它支持多种消息传递模式，常见的包括：

1. 请求-响应模式（REQ-REP）：客户端发送请求，服务器响应请求。客户端和服务器之间保持同步。
2. 发布-订阅模式（PUB-SUB）：发布者发布消息，多个订阅者接收消息，发布者和订阅者之间是异步的。
3. 管道模式（PUSH-PULL）：生产者向工作者发送消息，工作者处理并返回结果，适合用于负载均衡。

在 ZeroMQ 中，所有的通信都是通过套接字（socket）实现的，套接字有不同的类型，每种类型对应不同的消息模式。

而 cppzmq 是一个 C++ 库，它是 ZeroMQ（一个高性能异步消息库）的 C++ 接口封装。ZeroMQ 是一个消息队列库，设计用于高吞吐量、低延迟的消息传递系统，广泛用于分布式计算、并行处理、实时系统等应用场景。cppzmq 为 C++ 开发者提供了一个更为面向对象和易于使用的接口。

1.2 主要特点：

- 1、面向对象：cppzmq 封装了 ZeroMQ 的核心 API，提供了如 socket, context, message 等类。
- 2、简化的接口：ZeroMQ 本身是一个相对低级别的库，直接使用时可能需要处理许多细节（例如，内存管理、资源释放等），而 cppzmq 简化了这些操作，减少了复杂性。
- 3、高效的异步通信：ZeroMQ 提供了高效的异步消息传递功能，支持多种消息队列模式，例如发布-订阅（pub-sub）、请求-响应（req-rep）、管道（push-pull）等。
- 4、跨平台：ZeroMQ（以及 cppzmq）是跨平台的，支持多种操作系统，可以在这些平台上实现高效的消息传递。
- 5、支持多种传输协议：ZeroMQ 支持多种传输协议，如 TCP、IPC、PIP（process-internal communication）和 multicast。cppzmq 继承了这一特性，可用于各种分布式系统中。
- 6、内存管理：cppzmq 封装了 ZeroMQ 的消息传递和内存管理，减少了内存泄漏的风险。

1.3 安装 `cppzmq`

可以通过以下方式安装 `cppzmq`：

代码块

```
1  sudo apt install libzmq3-dev # ZeroMQ 库
2  sudo apt install libcppzmq-dev # cppzmq 库
3
4  # 若手动安装，克隆 GitHub 仓库: cppzmq GitHub
```

2 ZMQ入门

2.1 cppzmq 的基本类与概念

1. `zmq::context_t`：代表 ZeroMQ 的上下文，负责管理套接字、消息的生命周期。程序通常只需一个上下文。
2. `zmq::socket_t`：ZeroMQ 套接字类，所有消息的发送和接收都通过它来完成。套接字的类型（例如，ZMQ_REQ，ZMQ_REP，ZMQ_PUB，ZMQ_SUB）定义了它的行为。
3. `zmq::message_t`：消息类，用于表示通过 ZeroMQ 发送和接收的数据。

2.2 示例代码一

示例代码一：Hello World 使用 cppzmq 实现一个基本的请求-响应模式（REQ-REP）

服务器端

```

1  #include <zmq.hpp>
2  #include <iostream>
3
4  int main() {
5      // 创建一个上下文
6      zmq::context_t context(1);
7
8      // 创建一个 REP 类型的套接字
9      zmq::socket_t socket(context, ZMQ_REP);
10
11     // 绑定端口
12     socket.bind("tcp://*:5555");
13
14     while (true) {
15         // 接收客户端的消息
16         zmq::message_t request;
17         socket.recv(request, zmq::recv_flags::none);
18
19         std::string received_message(static_cast<char*>(request.data()),
request.size());
20         std::cout << "Received request: " << received_message << std::endl;
21
22         // 发送响应
23         zmq::message_t reply(5);
24         memcpy(reply.data(), "World", 5);
25         socket.send(reply, zmq::send_flags::none);
26     }
27
28     return 0;
29 }
30

```

客户端

```

代码块
1  #include <zmq.hpp>
2  #include <iostream>
3
4  int main() {
5      // 创建一个上下文
6      zmq::context_t context(1);
7
8      // 创建一个 REQ 类型的套接字
9      zmq::socket_t socket(context, ZMQ_REQ);
10

```

```

11 // 连接到服务器
12 socket.connect("tcp://localhost:5555");
13
14 // 发送请求
15 zmq::message_t request(5);
16 memcpy(request.data(), "Hello", 5);
17 socket.send(request, zmq::send_flags::none);
18
19 // 接收响应
20 zmq::message_t reply;
21 socket.recv(reply, zmq::recv_flags::none);
22 std::string received_message(static_cast<char*>(reply.data()),
reply.size());
23 std::cout << "Received reply: " << received_message << std::endl;
24
25 return 0;
26 }

```

CMakeLists

代码块

```

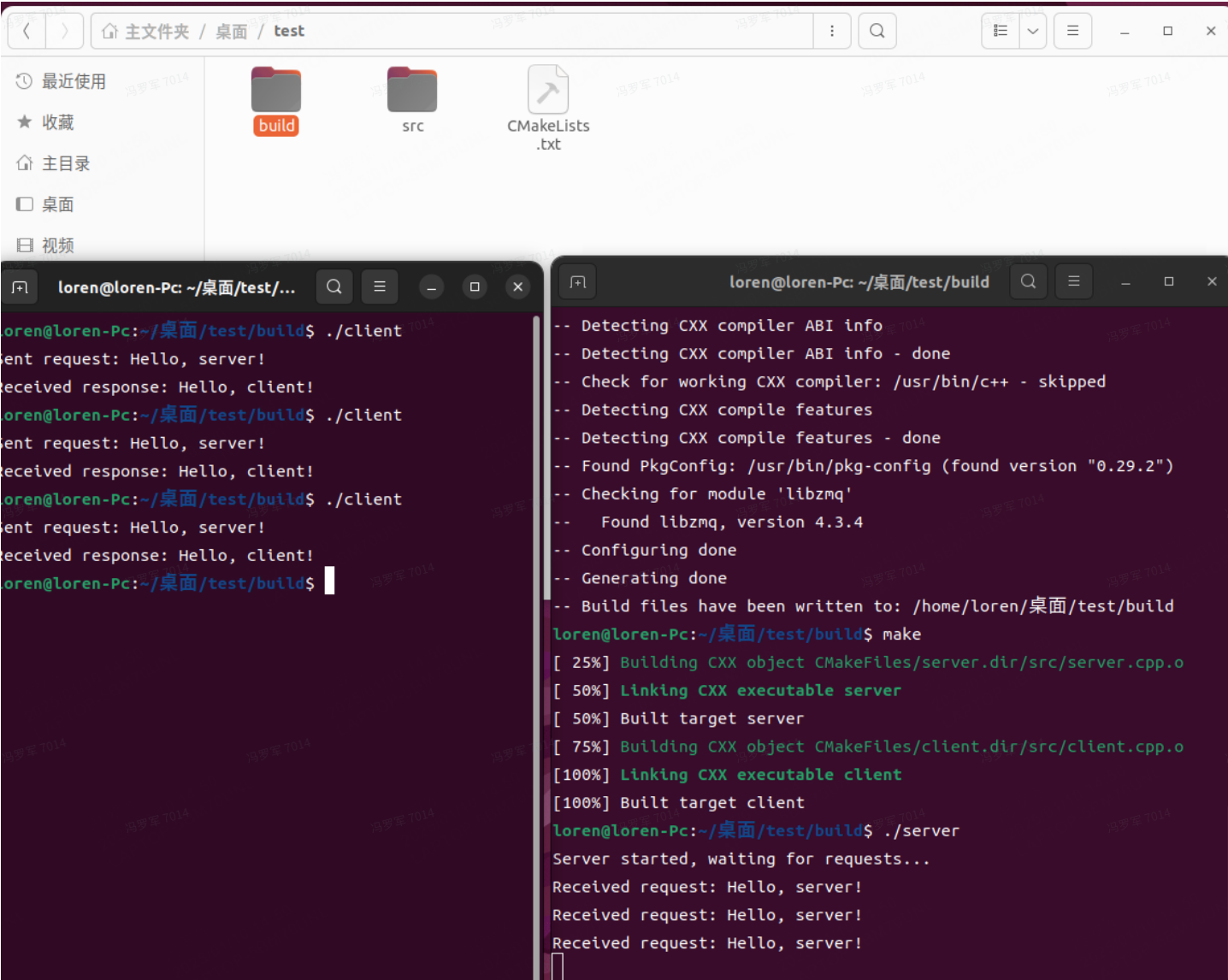
1 cmake_minimum_required(VERSION 3.10)
2 project(ZmqExample)
3
4 # 设置 C++ 标准
5 set(CMAKE_CXX_STANDARD 11)
6
7 # 查找 ZeroMQ 库
8 find_package(PkgConfig REQUIRED)
9 pkg_check_modules(ZMQ REQUIRED libzmq)
10
11 # 包含 ZeroMQ 头文件
12 include_directories(${ZMQ_INCLUDE_DIRS})
13
14 # 创建可执行文件
15 add_executable(server src/server.cpp)
16 add_executable(client src/client.cpp)
17
18 # 链接 ZeroMQ 库
19 target_link_libraries(server ${ZMQ_LIBRARIES})
20 target_link_libraries(client ${ZMQ_LIBRARIES})

```

代码说明:

- 服务器端 (REP 类型套接字) 绑定在端口 5555 上, 等待来自客户端的请求。

- 客户端 (REQ 类型套接字) 向服务器发送一个 "Hello" 消息, 并等待回复。
- 服务器端收到请求后回复 "World", 客户端接收到并输出该消息。
- 这种模式是同步的, 客户端发送请求后会阻塞, 直到服务器回复响应。



2.3 示例代码二

示例代码二：发布-订阅模式 (PUB-SUB)

1. 发布者 (PUB)

代码块

```
1 #include <zmq.hpp>
2 #include <iostream>
3 #include <thread>
4
5 void publisher() {
```

```

6      zmq::context_t context(1);
7      zmq::socket_t publisher(context, ZMQ_PUB);
8      publisher.bind("tcp://*:5556");
9
10     while (true) {
11         zmq::message_t message(5);
12         memcpy(message.data(), "Hello", 5);
13         publisher.send(message, zmq::send_flags::none);
14         std::this_thread::sleep_for(std::chrono::seconds(1));
15     }
16 }
17
18 int main() {
19     std::thread pub_thread(publisher);
20     pub_thread.join();
21     return 0;
22 }

```

2. 订阅者 (SUB)

代码块

```

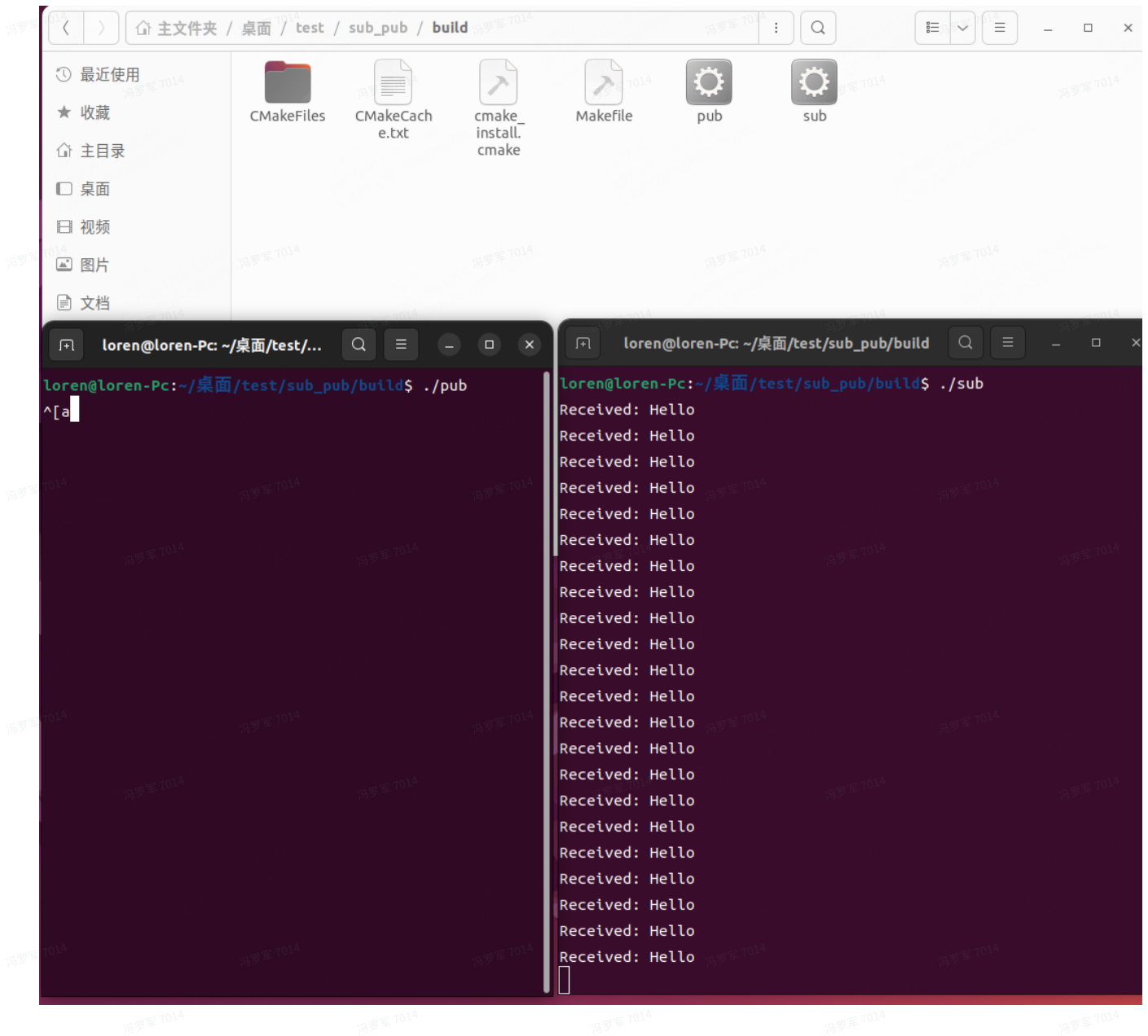
1  #include <zmq.hpp>
2  #include <iostream>
3
4  int main() {
5      zmq::context_t context(1);
6      zmq::socket_t subscriber(context, ZMQ_SUB);
7      subscriber.connect("tcp://localhost:5556");
8
9      // 订阅所有消息
10     subscriber.setsockopt(ZMQ_SUBSCRIBE, "", 0);
11
12     while (true) {
13         zmq::message_t message;
14         subscriber.recv(message, zmq::recv_flags::none);
15         std::string received_message(static_cast<char*>(message.data()),
16                                     message.size());
17         std::cout << "Received: " << received_message << std::endl;
18     }
19     return 0;
20 }

```

```
1 cmake_minimum_required(VERSION 3.10)
2 project(ZmqExample)
3
4 # 设置 C++ 标准
5 set(CMAKE_CXX_STANDARD 11)
6
7 # 查找 ZeroMQ 库
8 find_package(PkgConfig REQUIRED)
9 pkg_check_modules(ZMQ REQUIRED libzmq)
10
11 # 包含 ZeroMQ 头文件
12 include_directories(${ZMQ_INCLUDE_DIRS})
13
14 # 创建可执行文件
15 add_executable(pub src/pub.cpp)
16 add_executable(sub src/sub.cpp)
17
18 # 链接 ZeroMQ 库
19 target_link_libraries(pub ${ZMQ_LIBRARIES})
20 target_link_libraries(sub ${ZMQ_LIBRARIES})
```

代码说明：

- 发布者每秒发送一次 "Hello" 消息。
- 订阅者通过 `setsockopt` 设置了订阅主题（`ZMQ_SUBSCRIBE`），并接收所有的消息。



3 高阶使用技巧与最佳实践

- **线程安全：** `zmq::context_t` 和 `zmq::socket_t` 是线程不安全的。如果在多线程中使用它们，应该保证每个线程使用独立的套接字实例，或者使用互斥锁保护。
- **消息性能优化：** ZeroMQ 支持异步操作和消息批量处理，可以通过非阻塞发送/接收和消息池来提高性能。
- **连接管理：** ZeroMQ 自动处理重连机制，但你仍然可以通过设置 `zmq::socket_t` 的一些参数来优化连接行为。

二、Protobuf

1.学习Protobuf基础

代码块

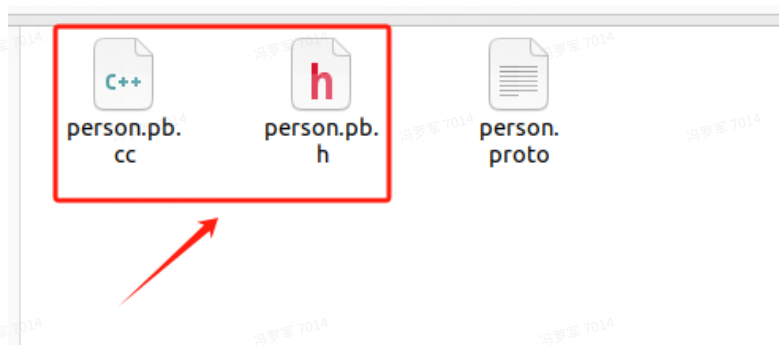
```
1 sudo apt update
2 sudo apt install protobuf-compiler
```

2.编写 .proto 文件：

Protobuf 使用 IDL 来定义消息类型。你需要创建person.proto 文件来描述你的数据结构。

代码块

```
1 syntax = "proto3";
2
3 message Person {
4     string name = 1;
5     int32 age = 2;
6 }
7
```



编译文件生成红框文件

代码块

```
1 protoc --cpp_out=. person.proto
```

编译main.cpp

代码块

```
1 #include <iostream>
```

```

2  #include <fstream>
3  #include "../proto/person.pb.h" // 引入 Protobuf 生成的头文件
4
5  int main() {
6      // 创建一个 Person 对象并设置值
7      Person person;
8      person.set_name("Alice");
9      person.set_age(30);
10
11     // 序列化到文件
12     std::ofstream output("person.dat", std::ios::binary);
13     if (!person.SerializeToOstream(&output)) {
14         std::cerr << "Failed to write person data to file." << std::endl;
15         return -1;
16     }
17     std::cout << "Person data serialized successfully!" << std::endl;
18
19     // 反序列化
20     Person new_person;
21     std::ifstream input("person.dat", std::ios::binary);
22     if (!new_person.ParseFromIstream(&input)) {
23         std::cerr << "Failed to parse person data." << std::endl;
24         return -1;
25     }
26     std::cout << "Deserialized Person: " << new_person.name() << ", Age: " <<
new_person.age() << std::endl;
27
28     return 0;
29 }

```

CMakeLists

代码块

```

1  cmake_minimum_required(VERSION 3.10)
2
3  # 设置项目名称和版本
4  project(ProtobufExample VERSION 1.0)
5
6  # 查找 Protobuf 库
7  find_package(Protobuf REQUIRED)
8
9  # 设置编译选项, 使用 C++ 11
10 set(CMAKE_CXX_STANDARD 11)
11
12 # 设置 Protobuf 生成的文件的源路径

```

```
13 set(PROTO_SRC_DIR ${CMAKE_SOURCE_DIR}/src)
14 set(PROTO_SRCS ${CMAKE_SOURCE_DIR}/proto)
15
16 # 将 Protobuf 生成的文件和主程序文件添加到目标中
17 add_executable(ProtobufExample ${PROTO_SRC_DIR}/main.cpp
    ${PROTO_SRCS}/person.pb.cc ${PROTO_SRCS}/person.pb.h)
18
19 # 链接 Protobuf 库
20 target_link_libraries(ProtobufExample ${Protobuf_LIBRARIES})
21
22 # 包含 Protobuf 头文件
23 target_include_directories(ProtobufExample PRIVATE ${Protobuf_INCLUDE_DIRS})
```

```
loren@loren-Pc:~/桌面/test/ProtobufExample/build$ ./ProtobufExample
Person data serialized successfully!
Deserialized Person: , Age: 0
```

三、结合 Protobuf 和 ZeroMQ使用

Protobuf 和 ZeroMQ 可一起使用，传输序列化的消息。你可以使用 ZeroMQ 进行消息传递，使用 Protobuf 进行数据的编码和解码。